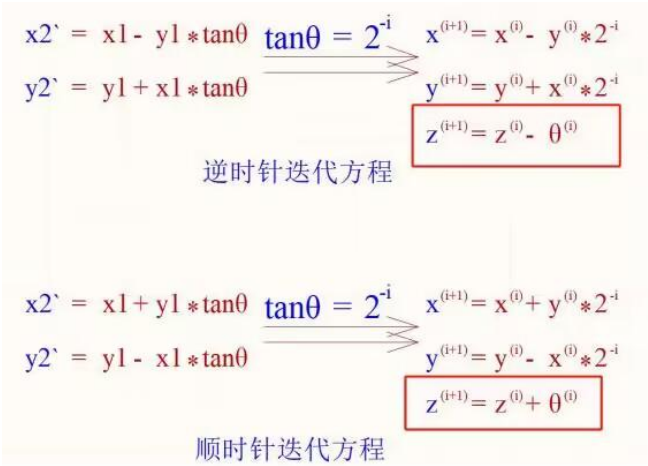
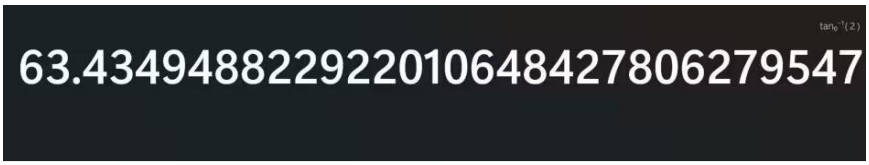
57-FOC 10

大家晚上好，欢迎参加张飞实战电子 FOC 第 11 期学习。
上节课我们讲了 Cordic 算法的迭代方程。

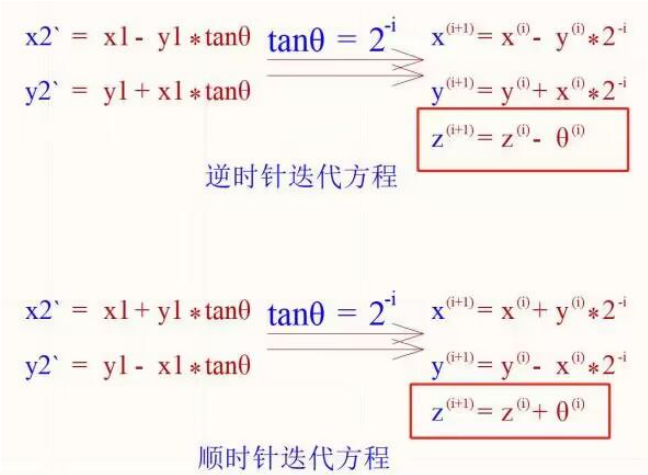


这里，角度累加器我们用 Z 来表示。再根据我们之前定义的角度正负，顺时针角度是正，逆时针角度是负。大家看下，这样的话，是不是从 i=0 开始的旋转角度，都会被加到角度累加器 z 中。通过这三个方程，只要迭代结束，就能拿到所有迭代次数的累加角度。那么这样的话，我们就把反正切求解角度的思路转换为迭代方程的计算了。

好，我们就来计算一个实例，让大家感受一下这个迭代过程。若 i = 0 时，x = 1，y= 2，我们根据迭代方程来计算计算，对应的结果是多少？ 或者，我们可以先用计算器把最终结果计算出来，arctan(2/1)是多少？



先算一下，后面再验证一下结果是否相符。计算出来呢，可以看到，最终的角度大致是 63.4349° 。



本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

现在我们根据这个迭代方程来计算一下，大家可以打开 excel，列一个计算表。这里呢，其实我们可以把两个方向的迭代方程整合为一个方程。



大家看下，上面的迭代方程，我们加入一个 d(i)旋转方向因子，就可以整合为一组方程。前面我们也讲过旋转方向是根据矢量的 y 坐标来确定的。y 大于 0，矢量顺时针旋转，y 小于 0，矢量逆时针旋转，目标都是要靠近 x 轴正半轴。所以，这些都理解之后，我们来看对应的实际计算，把迭代计算过程放到 EXCEL 中。

实例：x0 = 1，y0 = 2						
迭代次数(i)	x(i)	y(i)	2^(-i)	d(i)	θ(i)	z(i)
0	1	2	1	1	45	0
1	3	1	0.5	1	26.5651	45
2	3.5	-0.5	0.25	-1	14.0362	71.5651
3	3.625	0.375	0.125	1	7.12502	57.5288
4	3.67188	-0.0781	0.0625	-1	3.57633	64.6538
5	3.67676	0.15137	0.03125	1	1.78991	61.0775
6	3.68149	0.03647	0.01563	1	0.89517	62.8674
7	3.68206	-0.0211	0.00781	-1	0.44761	63.7626
8	3.68222	0.00771	0.00391	1	0.22381	63.315
9	3.68225	-0.0067	0.00195	-1	0.11191	63.5388
10	3.68227	0.00052	0.00098	1	0.05595	63.4269
11	3.68227	-0.0031	0.00049	-1	0.02798	63.4828
12	3.68227	-0.0013	0.00024	-1	0.01399	63.4548
13	3.68227	-0.0004	0.00012	-1	0.00699	63.4409

问：那初始位置不是（1，2）呢？
答：我们先讨论第 1 象限，后面会讨论 4 个象限。

大家看下这个表，第一次迭代，我们知道 i=0 的时候，对应的 x 跟 y 分别是 1 跟 2。此时呢，y 是大于 0 的，所以矢量要顺时针旋转，顺逆时针旋转的话，d 是+1。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

$$x^{(i+1)} = x^{(i)} + y^{(i)} * 2^{-i}$$
$$y^{(i+1)} = y^{(i)} - x^{(i)} * 2^{-i}$$
$$z^{(i+1)} = z^{(i)} + \theta^{(i)}$$

所以，第一次迭代的话，我们可以按照这个方程计算，这样的话，计算出来的新的 x = 3，新的 y = 1。那么，旋转之后呢，对应的角度累加器中的角度是 45° 。

问：45 度是怎么得来的？

答：$\tan\theta = 2^{-i}\ (i = 0,1,2,3,4.....)$，i=0， 角度是 45 度，也就是 tan45 = 1。

实例：x0 = 1，y0 = 2						
迭代次数(i)	x(i)	y(i)	2^(-i)	d(i)	θ(i)	z(i)
0	1	2	1	1	45	0
1	3	1	0.5	1	26.5651	45
2	3.5	-0.5	0.25	-1	14.0362	71.5651
3	3.625	0.375	0.125	1	7.12502	57.5288
4	3.67188	-0.0781	0.0625	-1	3.57633	64.6538
5	3.67676	0.15137	0.03125	1	1.78991	61.0775
6	3.68149	0.03647	0.01563	1	0.89517	62.8674
7	3.68206	-0.0211	0.00781	-1	0.44761	63.7626
8	3.68222	0.00771	0.00391	1	0.22381	63.315
9	3.68225	-0.0067	0.00195	-1	0.11191	63.5388
10	3.68227	0.00052	0.00098	1	0.05595	63.4269
11	3.68227	-0.0031	0.00049	-1	0.02798	63.4828
12	3.68227	-0.0013	0.00024	-1	0.01399	63.4548
13	3.68227	-0.0004	0.00012	-1	0.00699	63.4409

所以，第一次迭代计算完，x y 更新了，角度累加器中的角度也更新了。那么，下一次旋转的话，就需要再判断 y 的正负，来决定旋转方向。因为 y 是 1，仍然是大于 0 的，所以第二次旋转，矢量还会顺时针旋转。

实例：x0 = 1，y0 = 2						
迭代次数(i)	x(i)	y(i)	2^(-i)	d(i)	θ(i)	z(i)
0	1	2	1	1	45	0
1	3	1	0.5	1	26.5651	45
2	3.5	-0.5	0.25	-1	14.0362	71.5651
3	3.625	0.375	0.125	1	7.12502	57.5288
4	3.67188	-0.0781	0.0625	-1	3.57633	64.6538
5	3.67676	0.15137	0.03125	1	1.78991	61.0775
6	3.68149	0.03647	0.01563	1	0.89517	62.8674
7	3.68206	-0.0211	0.00781	-1	0.44761	63.7626
8	3.68222	0.00771	0.00391	1	0.22381	63.315
9	3.68225	-0.0067	0.00195	-1	0.11191	63.5388
10	3.68227	0.00052	0.00098	1	0.05595	63.4269
11	3.68227	-0.0031	0.00049	-1	0.02798	63.4828
12	3.68227	-0.0013	0.00024	-1	0.01399	63.4548
13	3.68227	-0.0004	0.00012	-1	0.00699	63.4409

所以，第二次旋转的话，旋转的角度还是正的。对应的旋转角度呢，是 26.5651° 。这个角度跟前一次旋转的角度累加，就得到第二次旋转的角度是 71.5651° 。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

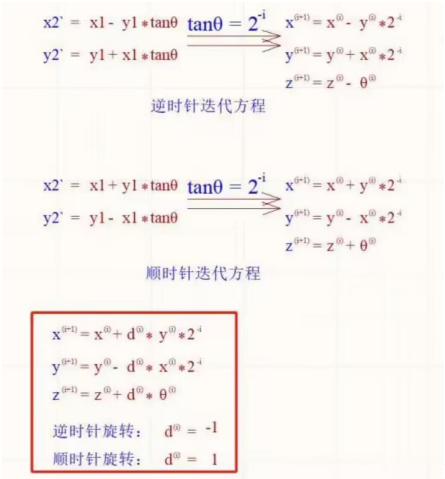
问：每个电机的初始位置不变吗？
答：计算反正切和电机没有任何关系，纯粹是计算数学函数。

实例：x0 = 1，y0 = 2						
迭代次数(i)	x(i)	y(i)	2^(-i)	d(i)	θ(i)	z(i)
0	1	2	1	1	45	0
1	3	1	0.5	1	26.5651	45
2	3.5	-0.5	0.25	-1	14.0362	71.5651
3	3.625	0.375	0.125	1	7.12502	57.5288
4	3.67188	-0.0781	0.0625	-1	3.57633	64.6538
5	3.67676	0.15137	0.03125	1	1.78991	61.0775
6	3.68149	0.03647	0.01563	1	0.89517	62.8674
7	3.68206	-0.0211	0.00781	-1	0.44761	63.7626
8	3.68222	0.00771	0.00391	1	0.22381	63.315
9	3.68225	-0.0067	0.00195	-1	0.11191	63.5388
10	3.68227	0.00052	0.00098	1	0.05595	63.4269
11	3.68227	-0.0031	0.00049	-1	0.02798	63.4828
12	3.68227	-0.0013	0.00024	-1	0.01399	63.4548
13	3.68227	-0.0004	0.00012	-1	0.00699	63.4409

问：就是第一个 x，y 是怎么确定的？
答：这个后面讲完反正切，当使用这个反正切函数的时候，你就会明白了。

问：这是函数输入值 XY，这里只是讲函数功能实现吧？
答：是的，和电机控制无关。

问：没有明白第二行数据怎么来的？



答：根据红框中的公式。

那么，第二次旋转完之后，y 小于 0 了，所以，必定第三次旋转就是逆时针旋转。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

实例：x0 = 1，y0 = 2						
迭代次数(i)	x(i)	y(i)	2^(-i)	d(i)	θ(i)	z(i)
0	1	2	1	1	45	0
1	3	1	0.5	1	26.5651	45
2	3.5	-0.5	0.25	-1	14.0362	71.5651
3	3.625	0.375	0.125	1	7.12502	57.5288
4	3.67188	-0.0781	0.0625	-1	3.57633	64.6538
5	3.67676	0.15137	0.03125	1	1.78991	61.0775
6	3.68149	0.03647	0.01563	1	0.89517	62.8674
7	3.68206	-0.0211	0.00781	-1	0.44761	63.7626
8	3.68222	0.00771	0.00391	1	0.22381	63.315
9	3.68225	-0.0067	0.00195	-1	0.11191	63.5388
10	3.68227	0.00052	0.00098	1	0.05595	63.4269
11	3.68227	-0.0031	0.00049	-1	0.02798	63.4828
12	3.68227	-0.0013	0.00024	-1	0.01399	63.4548
13	3.68227	-0.0004	0.00012	-1	0.00699	63.4409

逆时针旋转的话，d 就是-1。

$$x^{(i+1)} = x^{(i)} - y^{(i)} * 2^{-i}$$

$$y^{(i+1)} = y^{(i)} + x^{(i)} * 2^{-i}$$

$$z^{(i+1)} = z^{(i)} - \theta^{(i)}$$

所以，此时的迭代方程就要使用这个

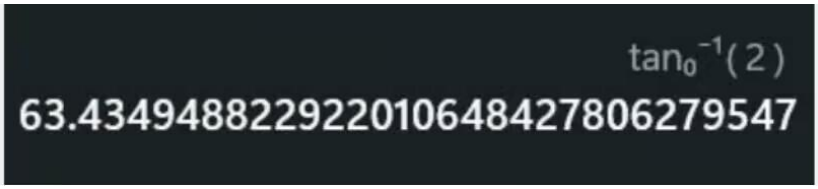
实例：x0 = 1，y0 = 2						
迭代次数(i)	x(i)	y(i)	2^(-i)	d(i)	θ(i)	z(i)
0	1	2	1	1	45	0
1	3	1	0.5	1	26.5651	45
2	3.5	-0.5	0.25	-1	14.0362	71.5651
3	3.625	0.375	0.125	1	7.12502	57.5288
4	3.67188	-0.0781	0.0625	-1	3.57633	64.6538
5	3.67676	0.15137	0.03125	1	1.78991	61.0775
6	3.68149	0.03647	0.01563	1	0.89517	62.8674
7	3.68206	-0.0211	0.00781	-1	0.44761	63.7626
8	3.68222	0.00771	0.00391	1	0.22381	63.315

这样的话，计算出来就是红框的值。 角度累加器中的角度，就是 71.5651° 减去 14.0362° ，这样的话也就是 57.5288° 。那么，就按照这个思路，只要 y 大于 0，就使用顺时针的迭代方程计算，只要 y 小于 0，就是用逆时针的迭代方程。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

实例：x0 = 1，y0 = 2						
迭代次数(i)	x(i)	y(i)	2^(-i)	d(i)	θ(i)	z(i)
0	1	2	1	1	45	0
1	3	1	0.5	1	26.5651	45
2	3.5	-0.5	0.25	-1	14.0362	71.5651
3	3.625	0.375	0.125	1	7.12502	57.5288
4	3.67188	-0.0781	0.0625	-1	3.57633	64.6538
5	3.67676	0.15137	0.03125	1	1.78991	61.0775
6	3.68149	0.03647	0.01563	1	0.89517	62.8674
7	3.68206	-0.0211	0.00781	-1	0.44761	63.7626
8	3.68222	0.00771	0.00391	1	0.22381	63.315
9	3.68225	-0.0067	0.00195	-1	0.11191	63.5388
10	3.68227	0.00052	0.00098	1	0.05595	63.4269
11	3.68227	-0.0031	0.00049	-1	0.02798	63.4828
12	3.68227	-0.0013	0.00024	-1	0.01399	63.4548
13	3.68227	-0.0004	0.00012	-1	0.00699	63.4409

最终，迭代了 13 次，求解出来的角度是 63.4409°，我们再拿这个结果，跟计算器计算的结果去对比。



两者之差大致是 0.006°，这个误差，完全能够满足我们的使用了。结合这个例子计算后，理解会不会更深刻一些？

实例：x0 = 1，y0 = 2						
迭代次数(i)	x(i)	y(i)	2^(-i)	d(i)	θ(i)	z(i)
0	1	2	1	1	45	0
1	3	1	0.5	1	26.5651	45
2	3.5	-0.5	0.25	-1	14.0362	71.5651
3	3.625	0.375	0.125	1	7.12502	57.5288
4	3.67188	-0.0781	0.0625	-1	3.57633	64.6538
5	3.67676	0.15137	0.03125	1	1.78991	61.0775
6	3.68149	0.03647	0.01563	1	0.89517	62.8674
7	3.68206	-0.0211	0.00781	-1	0.44761	63.7626
8	3.68222	0.00771	0.00391	1	0.22381	63.315
9	3.68225	-0.0067	0.00195	-1	0.11191	63.5388
10	3.68227	0.00052	0.00098	1	0.05595	63.4269
11	3.68227	-0.0031	0.00049	-1	0.02798	63.4828
12	3.68227	-0.0013	0.00024	-1	0.01399	63.4548
13	3.68227	-0.0004	0.00012	-1	0.00699	63.4409

问：为什么迭代 13 次就算 OK 了，迭代结束的原则是什么？
答：其它次数也可以，看你能够容忍的偏差。

问：迭代停止的条件是什么？
答：角度在精度可接受的范围内。

问：硬件 cordic 一般误差是多大？
答：这要看硬件是具体如何设计的。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

实际上，大家看下，随着迭代次数的增多，迭代方程中的累加值偏差都已经变得很小很小了。特别是累加的角度值，都已经是零点几度以内了。越往后，角度值只是在增加精度。但是呢，我们肯定不能任凭这样迭代下去，在精度可接受的范围内，就可以让迭代结束。毕竟，迭代次数多了，也是耗费时间的，在精度允许的情况下，我们让迭代次数小一些，耗费的时间就短一些。 迭代次数少些，算法的时间复杂度也就小一些，更能适合低性能的单片机。

大家也可以自己多举几个其它的例子，自己感受一下计算过程。当迭代到第 10 次的时候，再往下去迭代，x 已经基本不变了，旋转角度也变得很小很小了。所以呢，实际上整个旋转的过程，我们完全可以取旋转 10 次后的角度累加器中的角度作为目标角度。当然，如果运算能力强大的话，也可以多取几次。就是说，10 次迭代的话，对应的结果是可以满足我们的使用场景的。不管是在低性能单片机，还是在稍微高性能的单片机中，都可以。所以呢，整个算法的实现，其实我们就可以取迭代的次数是 10 次，这里我们达成一致意见啊。

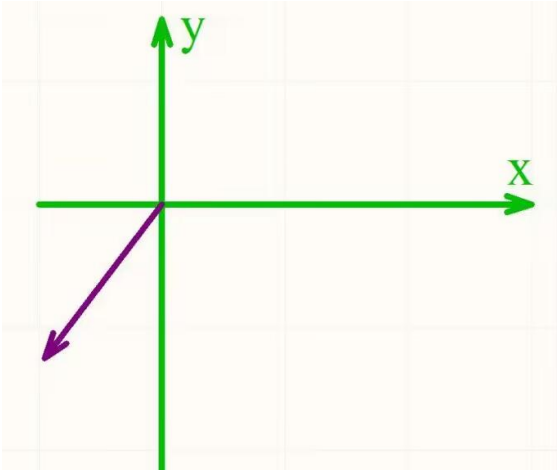
$$x2` = x1 - y1 * \tan\theta \quad \tan\theta = 2^{-i} \quad x^{(i+1)} = x^{(i)} - y^{(i)} * 2^{-i}$$
$$y2` = y1 + x1 * \tan\theta \quad\quad y^{(i+1)} = y^{(i)} + x^{(i)} * 2^{-i}$$
$$z^{(i+1)} = z^{(i)} - \theta^{(i)}$$

逆时针迭代方程

$$x2` = x1 + y1 * \tan\theta \quad \tan\theta = 2^{-i} \quad x^{(i+1)} = x^{(i)} + y^{(i)} * 2^{-i}$$
$$y2` = y1 - x1 * \tan\theta \quad\quad y^{(i+1)} = y^{(i)} - x^{(i)} * 2^{-i}$$
$$z^{(i+1)} = z^{(i)} + \theta^{(i)}$$

顺时针迭代方程

然后大家再来看，就目前来说，我们推导出来的这些迭代方程，是不是能够满足所有的旋转场景了呢？我举个例子，如果矢量在第二象限或者第三象限，大家想想看，我们这样去旋转，能不能求解出正确的角度啊？那么能不能求解出正确的角度，其实我们可以推导出来。刚才我们已经把算法旋转的次数定下来了，是确定的 10 次 。或者说，我们多旋转几次，让次数是 13 次，此时呢矢量又是在第二象限或者第三象限，要推导一下经过 13 次旋转，能不能把正确角度求解出来。这里呢，我们可以一起来计算一下。



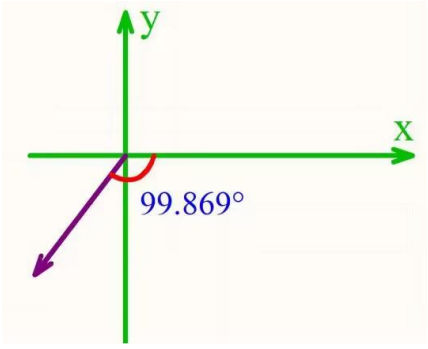
大家看下，若矢量此时处于第三象限，因为最终目的是要旋转它靠近 x 正半轴。所以呢，能够得到最少的旋转次数的旋转方向就是逆时针旋转。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

θ(i)
45
26.5651
14.0362
7.12502
3.57633
1.78991
0.89517
0.44761
0.22381
0.11191
0.05595
0.02798
0.01399

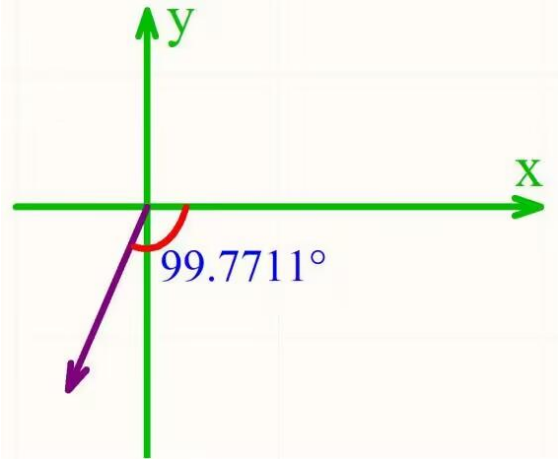
问：可以插入一个旋转 90 度的，用于适配第二第三象限么？
答：是的，后面就要这么做。

我们来看下每次旋转的角度值。如果说一个矢量始终往一个方向旋转，那么 13 次旋转结束后，最终的角度值就是上图 13 个角度的累加是吧？所以，我们能够得出正确的角度的情况是，一个矢量始终往靠近 x 正半轴的方向旋转，旋转 13 次，对应的极限角度就是 13 次角度的累加。13 次累加起来的话也就对应 99.869°。

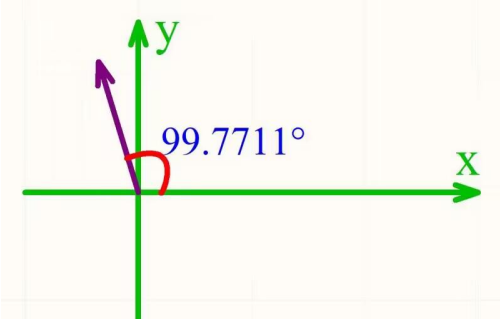


所以，如果我们旋转 13 次算法正好结束，那么矢量只能处于第三象限的当前位置。

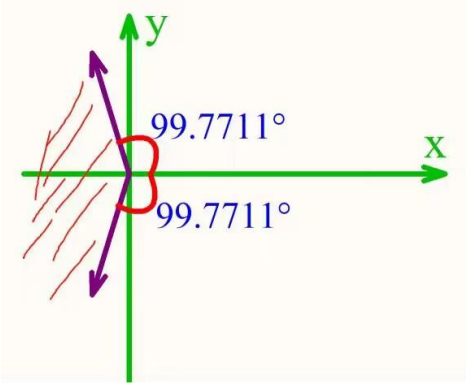
本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



而如果我们只旋转 10 次的话，那么矢量只能处于第三象限的当前位置。

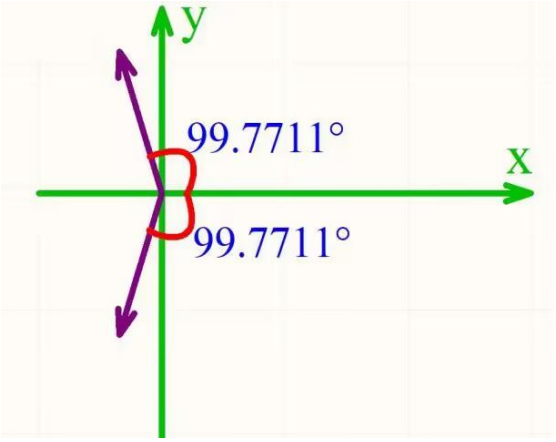


同理呢，矢量处于第二象限也是一样的。这样的话，也就意味着我们的算法适应能力不是很强，还有一些情况没有考虑到。



所以呢，我们也要想办法解决矢量在红色阴影部分的情况。如果我们判断知道矢量在第二象限或者第三象限，我们就先旋转一次，把矢量旋转到第一象限或者第四象限。只要矢量被旋转到了第一象限　或者第四象限，是不是我们目前的迭代方程就可以直接迭代得到结果了啊？刚才有同学说第一次旋转可以先转个 90° 。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



实际上大家看，象限跟象限的角度间隔都是 90° 。所以，只要在第二象限或者第三象限内，我们旋转 90° ，必定可以旋转到第一象限或者第四象限。那么，既然转 90° 可以满足我们的要求，转 90° 怎么实现呢？实际上很容易。

$$\begin{aligned} x2 &= x1*\cos\theta + y1*\sin\theta \\ y2 &= y1*\cos\theta - x1*\sin\theta \end{aligned} \qquad \begin{aligned} x^{(i+1)} &= x^{(i)}*\cos\theta^{(i)} + y^{(i)}*\sin\theta^{(i)} \\ y^{(i+1)} &= y^{(i)}*\cos\theta^{(i)} - x^{(i)}*\sin\theta^{(i)} \end{aligned}$$

顺时针旋转

$$\begin{aligned} x2 &= x1*\cos\theta - y1*\sin\theta \\ y2 &= y1*\cos\theta + x1*\sin\theta \end{aligned} \qquad \begin{aligned} x^{(i+1)} &= x^{(i)}*\cos\theta^{(i)} - y^{(i)}*\sin\theta^{(i)} \\ y^{(i+1)} &= y^{(i)}*\cos\theta^{(i)} + x^{(i)}*\sin\theta^{(i)} \end{aligned}$$

逆时针旋转

大家来看啊，之前我们花时间推导了平面直角坐标系内的旋转方程。要转 90° 的话，其实就是把上述方程中的 θ ，替换为 90° 。这样的话，就可以得到旋转 90° 之后的坐标了。

$$\begin{aligned} x2 &= x1*\cos\theta + y1*\sin\theta \\ y2 &= y1*\cos\theta - x1*\sin\theta \end{aligned} \qquad \begin{aligned} x^{(i+1)} &= x^{(i)}*\cos\theta^{(i)} + y^{(i)}*\sin\theta^{(i)} \\ y^{(i+1)} &= y^{(i)}*\cos\theta^{(i)} - x^{(i)}*\sin\theta^{(i)} \end{aligned}$$

平面坐标系顺时针旋转方程

$$\begin{aligned} x2 &= x1*\cos\theta - y1*\sin\theta \\ y2 &= y1*\cos\theta + x1*\sin\theta \end{aligned} \qquad \begin{aligned} x^{(i+1)} &= x^{(i)}*\cos\theta^{(i)} - y^{(i)}*\sin\theta^{(i)} \\ y^{(i+1)} &= y^{(i)}*\cos\theta^{(i)} + x^{(i)}*\sin\theta^{(i)} \end{aligned}$$

平面坐标系逆时针旋转方程

$$\begin{aligned} x^{(i+1)} &= x^{(i)}*\cos\theta^{(i)} + y^{(i)}*\sin\theta^{(i)} \\ y^{(i+1)} &= y^{(i)}*\cos\theta^{(i)} - x^{(i)}*\sin\theta^{(i)} \end{aligned}$$

$$\begin{aligned} x^{(i+1)} &= x^{(i)}*0 + y^{(i)}*1 \\ y^{(i+1)} &= y^{(i)}*0 - x^{(i)}*1 \end{aligned} \qquad \begin{aligned} x^{(i+1)} &= x^{(i)}*0 - y^{(i)}*1 \\ y^{(i+1)} &= y^{(i)}*0 + x^{(i)}*1 \end{aligned}$$

大家看，因为 cos90° = 0，sin90° = 1，所以这样就可以知道旋转后的 x y 跟旋转前 x y 的关系。

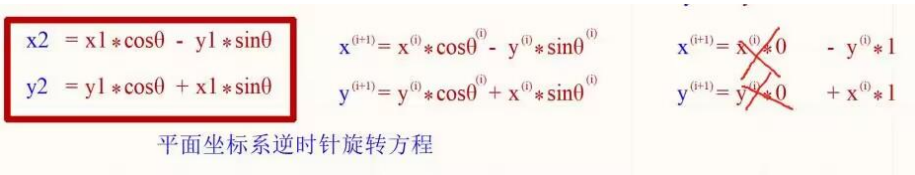
$$\begin{aligned} x2 &= x1*\cos\theta + y1*\sin\theta \\ y2 &= y1*\cos\theta - x1*\sin\theta \end{aligned} \qquad \begin{aligned} x^{(i+1)} &= x^{(i)}*\cos\theta^{(i)} + y^{(i)}*\sin\theta^{(i)} \\ y^{(i+1)} &= y^{(i)}*\cos\theta^{(i)} - x^{(i)}*\sin\theta^{(i)} \end{aligned}$$

平面坐标系顺时针旋转方程

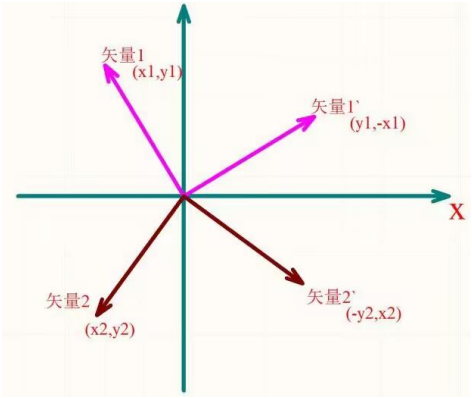
$$\begin{aligned} x^{(i+1)} &= x^{(i)}*0 + y^{(i)}*1 \\ y^{(i+1)} &= y^{(i)}*0 - x^{(i)}*1 \end{aligned}$$

也就是说，顺时针旋转 90° ，对应的坐标关系就是 旋转后的 x 就是旋转前的 y，旋转后的 y 就是旋转前的-x。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



逆时针旋转 90°，对应的坐标关系就是　旋转后的 x 就是旋转前的-y，旋转后的 y 就是旋转前的 x。 所以，要实现旋转 90°，只需要把坐标变换一下就可以实现。这个方法的话，是不是比较简单的啊，这样就轻松地实现了旋转 90 度。



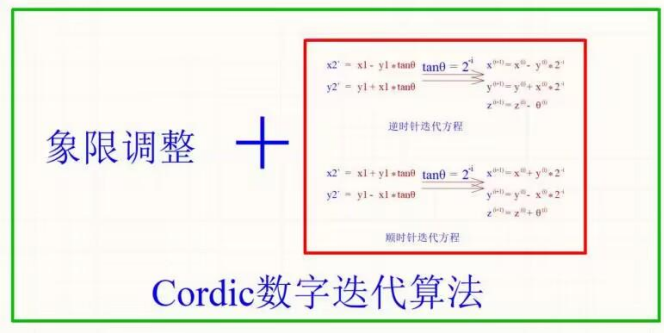
对应的旋转关系呢，可以用这幅图来表示。这个旋转 90° 的关系，大家可以理解的吧？

上面的旋转思想和过程，大家一定要明白，不然后面的代码就是天书了。有点迷糊的可以自己多推导过程，多举几个例子，试着旋转，这步解决了，后面代码基本也就掌握了。

问：判断完是第二第三象限，直接把 90 度带进去吗？还是自动加 90 度再计算？
答：是的，坐标按这个式子变换，把这个 90 度加到累加器 Z 中。

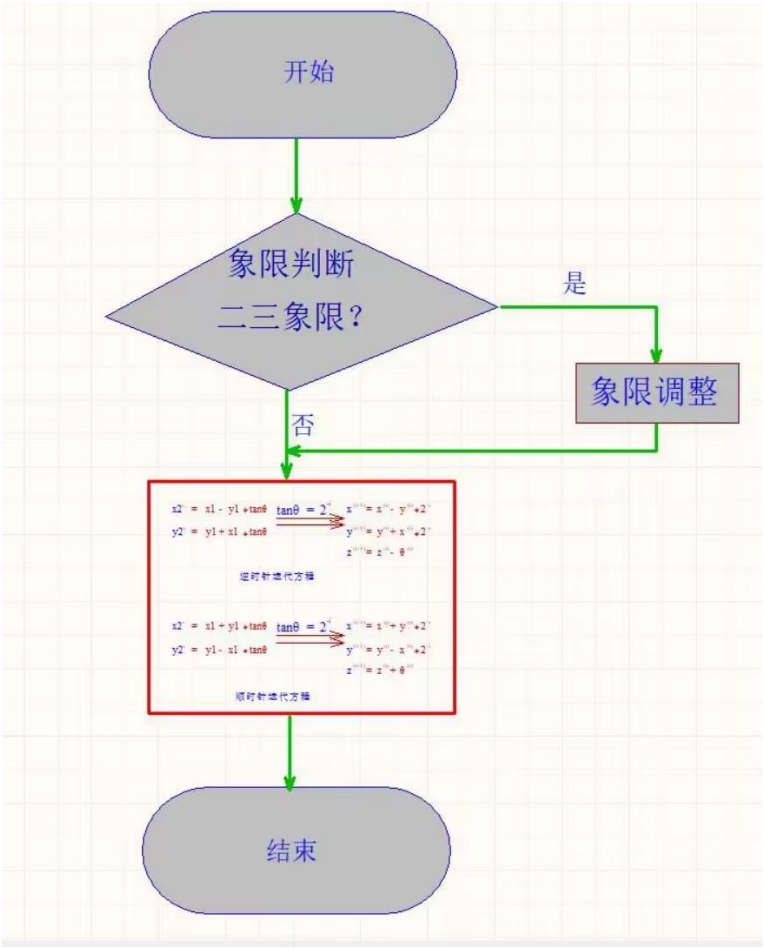
问：怎么判断是在第二，第三象限？
答：第二象限，x<0，　y>0，第三象限，x<0，　y<0。

处理后的算法框图是这样。



那么，这样处理之后呢，算法的适应性就变广了。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



那么，到目前为止的话，我们可以大致先画出这样一幅简要的算法流程图。算法一开始，就判断象限，决定是否要进行坐标的调整。如果不调整的话，就直接计算迭代方程，等到迭代结束后，就把对应的角度取出来，就得到要求解的角度了。

当然，在迭代的过程中，还要考虑迭代的结束啊，这里是简化版的流程。然后我们接着来看，这个算法的话，要编程落地。一些落地前的准备工作需要做好。例如，我们每次旋转的角度，需要存在单片机的 Flash 中，然后，每次旋转要查表获取旋转的角度，然后累加到角度累加器中， 既然要查表的话，就要把角度表给做好。

) = 1, y0 = 2

2^(-i)	d(i)	θ(i)	z(i)
1	1	45	0
0.5	1	26.5651	45
0.25	-1	14.0362	71.5651
0.125	1	7.12502	57.5288
0.0625	-1	3.57633	64.6538
0.03125	1	1.78991	61.0775
0.01563	1	0.89517	62.8674
0.00781	-1	0.44761	63.7626
0.00391	1	0.22381	63.315
0.00195	-1	0.11191	63.5388
0.00098	1	0.05595	63.4269
0.00049	-1	0.02798	63.4828
0.00024	-1	0.01399	63.4548
0.00012	-1	0.00699	63.4409

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

做表的话，其实就是把这一列角度值转换为实际使用的值。在程序中表示的时候，角度要归一化，然后用 Q15 格式来表示。之前我们也讲过，-180°~180° 的角度值我们是用 -32768~32767 的数据来表示的。

问：为什么要-180°~180° 这一列的角度多为正啊？
答：其实是一样的，Q15 和 Q16 表达角度，二进制数是一样的，你可以随便找个角度计算一下，看看二进制数是否几乎一样。

问：为什么用这个 cordic 呀？
答：为了通过反电动势得到转子角度。

问：是不是所有的单片机计算都可以通过归一法换成 Q15 或 Q16 的格式来算啊？
答：大多数可以。

这样的话，我们就把上面的角度值对应的 Q15 值给计算出来。计算的方法呢，就是把这一列的数据除以 180，然后乘以 32768。 θ(i)/180 * 32768 这是计算的公式。

今天的内容就先讲这么多。
大家早点休息，谢谢大家，晚安！

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。